ROS 2 basics cheatsheet

Commands from lessons A0 to A7 and B1 (ROS 2 Jazzy). Words in <angle brackets> are placeholders: replace them, without the brackets. Text in quotes like "{data: hello}" is YAML: field names and their values, as the message, request or goal defines them (ros2 interface show <type> lists the fields). Course: https://github.com/Turgibot/ros2-tutorial

Terminal and environment A0

`source /opt/ros/jazzy/setup.bash`
Make ROS 2 available in this terminal (already done in the course container)

`source install/setup.bash`
Make your own built workspace available (run in the workspace folder, after a build)

`echo $ROS_DOMAIN_ID`
Show the domain; programs only see each other in the same domain (default 0)

`export ROS_DOMAIN_ID=5`
Use domain 5 in this terminal (0 to 101)

`Ctrl+C`
Stop the program running in the terminal. (Ctrl+Z only suspends it: do not use it.)

`Tab`
Complete a command, a topic name or a type. Press twice to list the choices

Nodes A1

`ros2 run <package> <executable>`
Start a program (a node) from a package

`ros2 run <package> <executable> --ros-args -r __node:=<new_name>`
Start it under another node name (run two copies)

`ros2 node list`
List the running nodes

`ros2 node info /<node>`
Show a node's topics, services, actions and parameters

Topics A1

`ros2 topic list`
List the topics (add -t to show their types)

`ros2 topic type /<topic>`
Show a topic's message type

`ros2 topic info /<topic>`
Type, publishers and subscribers (add -v for their QoS settings)

`ros2 topic echo /<topic>`
Print the messages as they arrive

`ros2 topic echo /<topic> --once`
Print one message and stop

`ros2 topic echo /<topic> --field <field>`
Print only one field of each message

`ros2 topic hz /<topic>`
Measure how many messages arrive per second

`ros2 topic pub --once /chatter std_msgs/msg/String "{data: hello}"`
Publish one message. The YAML in quotes is field: value; this type has one field, data (see: ros2 interface show std_msgs/msg/String)

`ros2 topic pub -r 1 /chatter std_msgs/msg/String "{data: hello}"`
The same, once per second until Ctrl+C

`ros2 topic list --include-hidden-topics`
Also list hidden topics (the ones an action creates)

Messages, services and interfaces A2

`ros2 interface list`
List every message, service and action type

`ros2 interface package <package>`
List the types a package defines

`ros2 interface show <package>/msg/<Type>`
Show the fields of a message (use /srv/ or /action/ for the others)

`ros2 service list`
List the services (add -t to show their types)

`ros2 service type /<service>`
Show a service's type

`ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 2, b: 3}"`
Call a service. The YAML is the request: its fields (here a and b) are listed above the --- line of ros2 interface show

`ros2 service list --include-hidden-services`
Also list hidden services (the ones an action creates)

Actions A3

`ros2 action list`
List the actions (add -t to show their types)

`ros2 action info /<action>`
Show an action's type, servers and clients

`ros2 action send_goal /<action> <type> "{<field>: <value>}"`
Send a goal and wait for the result. The YAML is the goal: its fields are above the first --- line of ros2 interface show <type>

`ros2 action send_goal --feedback /<action> <type> "{<field>: <value>}"`
Same, and print the feedback while it runs

Parameters A4

`ros2 param list`
List the parameters of every node (or: ros2 param list /<node>)

`ros2 param get /<node> <name>`
Read a parameter

`ros2 param describe /<node> <name>`
Show its type, description and limits

`ros2 param set /<node> <name> <value>`
Change it while the node runs (lost when the node stops)

`ros2 param dump /<node>`
Print all of a node's parameters as YAML

`ros2 run <package> <executable> --ros-args -p <name>:=<value>`
Set a parameter when the node starts

`ros2 run <package> <executable> --ros-args --params-file <file.yaml>`
Set parameters from a YAML file (top-level key = node name)

Launch files A4

`ros2 launch <package> <file.launch.py>`
Start everything a launch file describes

`ros2 launch <package> <file.launch.py> <arg>:=<value>`
Give the launch file an argument

`ros2 launch <package> <file.launch.py> --show-args`
List its arguments and their defaults, without starting anything

Packages and workspaces A5

`ros2 pkg list`
List the installed packages

`ros2 pkg prefix <package>`
Show where a package is installed (add --share for its share folder)

`ros2 pkg executables <package>`
List the programs a package offers to ros2 run

`ros2 pkg create --build-type ament_python --node-name <node> <package>`
New Python package with one program (ament_cmake for C++)

`colcon build`
Build every package under src/ (run in the workspace folder)

`colcon build --packages-select <package>`
Build only one package

`colcon build --symlink-install`
Link Python, launch and YAML files instead of copying (edit without rebuilding)

`colcon list`
List the packages colcon found

`rosdep install --from-paths src --ignore-src -y`
Install the system packages the workspace needs

`echo $AMENT_PREFIX_PATH | tr : '\n'`
Show the workspaces in the search order, front first

Frames and TF2 A6

`ros2 run tf2_ros tf2_echo <from_frame> <to_frame>`
Print where <to_frame> is, as seen from <from_frame>

`ros2 run tf2_ros static_transform_publisher --x 0.1 --frame-id <parent> --child-frame-id <child>`
Publish a fixed frame (also --y --z --roll --pitch --yaw)

`ros2 run tf2_tools view_frames`
Draw the frame tree into frames_<date>.pdf

`ros2 run tf2_ros tf2_monitor`
Show, per frame, how long its transforms take to arrive

`ros2 topic echo /tf_static --once`
Look at the transform messages of fixed frames

`ros2 run rviz2 rviz2 -d <file.rviz>`
Open RViz with a saved configuration

ros2_control A7

`ros2 control list_controllers`
List the controllers and whether each is active (add -v for the interfaces each claims)

`ros2 control list_hardware_components`
List the hardware components (mock or real) and their interfaces

`ros2 control list_hardware_interfaces`
List every command and state interface, and which are claimed

`ros2 control switch_controllers --deactivate <controller> --activate <controller>`
Hand the joints from one controller to another

`ros2 topic echo /joint_states --once`
Read the joint names, angles, velocities and efforts

The Kinova arm

A6, A7, B1

```
ros2 launch kortex_description view_robot.launch.py gripper:=robotiq_2f_85
```

The arm in RViz with joint sliders (no controllers)

```
xacro $(ros2 pkg prefix kortex_description)/share/kortex_description/robots/kinova.urdf.xacro robot_ip:=xxx.yyy.zzz.www name:=kinova arm:=gen3 gripper:=robotiq_2f_85 dof:=7 > /tmp/kinova.urdf
```

Generate the arm's URDF from the xacro files

```
check_urdf /tmp/kinova.urdf
```

Check a URDF and print its tree of links

```
ros2 launch kortex_bringup gen3.launch.py robot_ip:=yyy.yyy.yyy.yyy use_fake_hardware:=true gripper:=robotiq_2f_85
```

The arm with controllers and fake hardware: the same launch file as the real arm

```
ros2 topic pub --once /joint_trajectory_controller/joint_trajectory trajectory_msgs/msg/JointTrajectory "{joint_names: [joint_1, joint_2, joint_3, joint_4, joint_5, joint_6, joint_7], points: [{positions: [0, 0, 0, 0, 0, 0, 0], time_from_start: {sec: 2}}]}"
```

Move all seven joints to zero in 2 seconds